

ONLINE RESORT MANAGEMENT & BOOKING SYSTEM

Exhaustive Technical Specification, End-to-End System Workflows, Database Architecture & API Reference

Document Title:	Bookme.com Comprehensive Technical System Manual
Version / Build:	v1.0.0 (Production Release — Turbopack Enabled)
Target Audience:	Software Engineers, System Architects, QA Engineers & DevOps
Core Web Framework:	Next.js 16 (App Router) + React 19 + TypeScript 5
Database & Persistence:	PostgreSQL via Prisma ORM v6 with Dynamic Transaction Support
Security & Auth:	NextAuth.js (JWT Provider) + Bcryptjs + Nodemailer Verification
Payments & Gateway:	Dodo Payments Hosted Checkout & Automated REST Refunds
Interactive Animations:	GSAP 3 (ScrollTrigger & ScrollTo) + Leaflet GIS Maps

Table of Document Contents:

1. Executive Summary & Vision	Section 6. Payment Gateway Integration
2. Full Technology Stack Deep-Dive	7. Role-Based Dashboard Desks
3. Database Schema & Data Models	8. Admin Console & RBAC Governance
4. Auth Subsystem & Registration	9. Complete API Endpoint Specification
5. Dynamic Tiered Cancellation Engine	10. Deployment & Environment Setup

1. Executive Summary & System Vision

Bookme.com is an enterprise-grade luxury resort management and guest booking web platform designed to streamline resort operations, deliver a high-end luxury guest experience, and manage multi-property inventory seamlessly. The application replaces fragmented legacy booking tools with an integrated Next.js 16 full-stack architecture featuring real-time invoice generation, automated payment processing, role-based staff operations, and a customer-centric dynamic cancellation policy engine.

Core Platform Objectives:

- **High-End Visual Aesthetics:** Deliver a luxury dark-mode visual experience (``bg-[#0C0A09]``) with GSAP scroll parallax, 3D mouse tilt cards, and interactive Leaflet map controls.
- **Resilient Guest Onboarding:** Provide friction-free registration with email verification codes, Nodemailer dev fallbacks, and account re-verification recovery.
- **Fair Cancellation Policy:** Guarantee a maximum fee cap of **10%** for late cancellations, providing guests with at least 90% refund security at all times.
- **Comprehensive Operations:** Support multi-role desks (Guest Portal, Staff Housekeeping Queue, Admin Governance) with granular RBAC permissions and audit CSV exports.

2. Technology Stack Deep-Dive

The application architecture leverages a modern, full-stack JavaScript/TypeScript toolchain built around Next.js 16 App Router. Below is an in-depth breakdown of each architectural layer:

Layer	Technology	Architectural Function & Rationale
Core Framework	Next.js 16 (Turbopack)	Utilizes the App Router paradigm with Server Components for fast initial page loads and API routes for backend execution. Turbopack provides sub-second hot module reloads.
Styling System	Tailwind CSS + Vanilla CSS	Tailwind CSS v4 CSS-first design system with dark glassmorphism tokens, backdrop blur filters (<code>`backdrop-blur-xl`</code>), custom scrollbars, and luxury amber accent glows.
Animation Engine	GSAP 3 (ScrollTrigger)	Drives hero mouse tilt physics, split-text letter reveals, directional panel transitions, timeline SVG path animations, and smooth section transitions.
GIS & Mapping	Leaflet.js + OpenStreetMap	Renders client-side interactive global maps with custom animated pulse markers. Automatically triggers smooth <code>`flyTo`</code> coordinate transitions when searching destinations.
Database Layer	Prisma ORM + PostgreSQL	Provides type-safe database queries, atomic database transactions (<code>`prisma.\$transaction`</code>), foreign key relationships, auto-generated migration histories, and seed data scripts.

Authentication	NextAuth.js + Bcryptjs	Manages JWT session encryption, password salting/hashing (10 rounds), credential verification, and attaches role & user type claims to session tokens.
Payment Gateway	Dodo Payments REST API	Handles hosted checkout session redirects, webhook security verification (`/api/webhooks/dodo`), and REST-based automated refunds (`POST /refunds`).
Mailer Dispatch	Nodemailer SMTP	Handles 6-digit email verification code dispatch, reservation booking confirmations, and detailed cancellation breakdown receipts. Features dev console fallback.

3. Complete Database Data Dictionary

The relational data model is defined in prisma/schema.prisma and enforced via PostgreSQL constraints. Below is the complete data dictionary detailing every entity, column, data type, and relation:

1. User Entity (`users` table):

Field Name	Type	Constraints	Description
id	String (UUID)	Primary Key, `@default(uuid())`	Unique user identifier
email	String	Unique, Indexed	Account login email address
password	String	Required	Bcrypt-hashed password string
fullName	String	Required	Full display name of user
role	Enum (RoleEnum)	Default: `GUEST`	Access level: `GUEST`, `STAFF`, `ADMIN`
isVerified	Boolean	Default: `false`	Indicates email verification status
verificationCode	String?	Nullable	6-digit email OTP code

2. Reservation Entity (`reservations` table):

Field Name	Type	Constraints	Description
id	String (UUID)	Primary Key	Unique reservation booking ID
checkIn	DateTime	Required	Scheduled arrival timestamp
checkOut	DateTime	Required	Scheduled departure timestamp
status	Enum	Default: `PENDING`	Status: `PENDING`, `CONFIRMED`, `CANCELED`, `CHECKED_IN`
totalAmount	Decimal(10,2)	Required	Grand total stay cost (room + add-ons)
guestId	String	Foreign Key (Guest)	Link to booking guest entity
roomId	String	Foreign Key (Room)	Link to assigned room unit

3. Payment Entity (`payments` table):

Field Name	Type	Constraints	Description
id	String (UUID)	Primary Key	Unique payment transaction ID
amount	Decimal(10,2)	Required	Payment currency amount
method	String	Required	Payment provider string e.g. 'Dodo Payments (ID: pay_123)'
status	Enum	Default: `PENDING`	Status: `PENDING`, `COMPLETED`, `FAILED`, `REFUNDED`
reservationId	String	Foreign Key (Reservation)	Link to parent reservation

4. Authentication & Security Subsystem

Security and user identity management are fundamental to Bookme.com. The application combines **NextAuth.js** for encrypted JWT session management with custom password salting, email verification OTP codes, and fallback handlers.

4.1 Registration Workflow (`POST /api/signup`):

- Input Validation:** Validates email format, password minimum length (6 chars), and full name.
- Existing Account Handling:** Checks if user exists. If account is verified (`isVerified = true`), returns HTTP 400 'Email already registered'.
- Unverified Account Recovery:** If user exists but is unverified (`isVerified = false`), the route updates the password hash, generates a fresh 6-digit OTP code, and re-dispatches the email code.
- Password Hashing:** Uses `bcryptjs` with salt round factor = 10.
- Transactional Provisioning:** Executes a Prisma `$transaction` creating User and Guest profile records simultaneously.

4.2 Email Verification & Mailer Resilience (`src/lib/mailer.ts`):

Nodemailer is configured to dispatch HTML email verification codes. If SMTP credentials (`SMTP_USER` / `SMTP_PASS`) are unconfigured or fail (e.g. in local offline dev mode), the mailer gracefully catches the error and logs the verification code directly to the server terminal, allowing registration testing without external email dependencies.

4.3 NextAuth Credentials & Authorization Sessions (`src/lib/auth.ts`):

NextAuth is configured with the Credentials Provider. Upon authentication:

- `authorize()` checks user existence, verifies password hash with `bcrypt.compare()`, and checks `isVerified === true`.
- Token callback injects `user.role` (`GUEST`, `STAFF`, `ADMIN`) and `user.type` into the encrypted JWT token.
- Session callback exposes `session.user.role` and `session.user.id` to client components and backend API endpoints.

5. Dynamic Tiered Cancellation & Refund Engine

To provide guest satisfaction while protecting resort revenue, Bookme.com implements a **Dynamic Tiered Cancellation Policy** enforcing a maximum fee cap of **10%**. Guests can cancel any stay at any time, with refund percentages dynamically calculated based on check-in notice:

Policy Tier	Notice Period	Refund %	Fee Retained	Operational Guarantee
Tier 1: Full Refund	≥ 7 Days before Check-In	100% Refund	0% (\$0.00)	Complete money-back guarantee with zero penalty.
Tier 2: Light Fee	3 to 7 Days before Check-In	95% Refund	5% Fee	Minor processing fee retained to cover gateway costs.
Tier 3: Max Fee Cap	< 3 Days / Same-Day / Past	90% Refund	10% Max Fee	Capped at 10% maximum fee; guest receives 90% refund.

5.1 Interactive Cancellation Modal Workflow (`src/app/dashboard/page.tsx`):

1. User clicks **Cancel Booking** on any reservation card in the Guest Portal.
2. Front-end calculates `daysRemaining = (checkInTime - Date.now()) / 24h``.
3. Modal renders a visual Policy Tier Badge (``100% Full Refund``, ``95% Refund (5% Fee)``, ``90% Refund (10% Max Fee)``).
4. Displays a financial summary table: Total Paid, Fee Deduction (-\$), Net Refund to Source Card (\$).
5. Enforces explicit user consent via agreement checkbox (``[x] I have read and accept the cancellation fee policy terms.``).

5.2 Backend Refund Execution (`DELETE /api/book`):

1. Authorizes guest ownership of reservation ID (`reservation.guestId === session.user.id``).
2. Computes exact ``refundAmount = totalAmount * (refundPercent / 100)`` and ``retentionFee = totalAmount - refundAmount``.
3. Extracts Dodo Payment ID string and dispatches POST refund to Dodo API (``https://test.dodopayments.com/refunds``).
4. Executes database transaction updating reservation ``status: 'CANCELED'`` and payment ``status: 'REFUNDED'``.
5. Dispatches HTML receipt email to guest with financial breakdown.

6. Dodo Payments Gateway Integration

Bookme.com integrates with **Dodo Payments API** for credit card processing and automated REST refunding. The checkout lifecycle operates across three primary stages:

1. Checkout Session Creation (`POST /api/checkout/[id]/session``):

- Reads grand total invoice amount (room base cost + add-on experiences + 15% resort taxes).
- Converts total amount to integer cents (`amountInCents = Math.round(totalAmount * 100)``).
- Sends POST request to `https://test.dodopayments.com/checkouts`` with product details and return URL.
- Returns hosted checkout URL string (`data.checkout_url``) to client for immediate redirect.

2. Payment Webhook Processor (`POST /api/webhooks/dodo``):

- Receives asynchronous payment status webhooks from Dodo Payments.
- On `payment.succeeded`` event, updates reservation status from `PENDING`` to `CONFIRMED`` and payment record status to `COMPLETED``.
- Generates room assignment (`room.status = 'OCCUPIED``) and dispatches booking confirmation email.

7. Role-Based Dashboard Desks

The dashboard (`src/app/dashboard/page.tsx``) renders dynamic role-specific interfaces based on `session.user.role``:

7.1 Guest Portal View:

- **My Bookings Tab:** Displays active and historical stay cards with status badges (`CONFIRMED``, `PENDING``, `CANCELED``), stay dates, suite categories, and 'Cancel Booking' triggers.
- **Saved Favorites Tab:** Grid view of bookmarked resorts with high-resolution imagery, location tags, ratings, and instant booking triggers.
- **My Profile Tab:** User account summary card showing verification badge, total bookings count, and saved resorts count.

7.2 Staff Task Queue & Housekeeping View:

- **Housekeeping Desk:** Real-time room status grid (`CLEAN``, `DIRTY``, `IN_PROGRESS``, `INSPECTED``). Staff can toggle cleaning statuses with one click.
- **Guest Request Queue:** Lists active guest requests (room service, extra towels, shuttle requests) with status filters and staff assignment selectors.

8. Admin Console & RBAC Governance

Administrative operators (`role === 'ADMIN``) have access to system-wide management controls:

- **Granular RBAC Permissions:** Custom operational roles can be provisioned with specific permission keys (`MANAGE_BOOKINGS``, `HOUSEKEEPING``, `FINANCE_ACCESS``, `STAFF_MANAGEMENT``, `DEPT_MANAGEMENT``, `MANAGE_SERVICES``, `PROPERTIES_MANAGEMENT``).

- **Staff Directory & Department Allocation:** Add new staff members, assign operational roles, and link them to resort departments.
- **Properties & Room Inventory:** Create new resort listings, edit suite categories, manage base night rates, and archive maintenance units.
- **Audit CSV Exports:** Embedded CSV generator buttons (`exportBookingsCSV`, `exportFinanceCSV`) for offline financial auditing.

9. Complete API Endpoint Specifications

Below is the exhaustive specification table for all 22 backend API endpoints:

HTTP	Endpoint Path	Auth Scope	Request Payload / Params	Response Summary & Function
GET	<code>`/api/resorts`</code>	Public	<code>`?page=1&limit;=12&query;=Bali`</code>	Returns paginated resort listings & total count
GET	<code>`/api/resorts/[id]`</code>	Public	URL param <code>`id`</code>	Returns resort detail, room types & services
POST	<code>`/api/signup`</code>	Public	<code>`{ email, password, fullName }`</code>	Hashes password, sends OTP code & returns user ID
POST	<code>`/api/signup/verify`</code>	Public	<code>`{ email, code }`</code>	Validates OTP code & sets <code>`isVerified = true`</code>
POST	<code>`/api/signup/resend`</code>	Public	<code>`{ email }`</code>	Generates & dispatches fresh 6-digit OTP code
POST	<code>`/api/auth/[...nextauth]`</code>	Public	<code>`{ email, password }`</code>	NextAuth JWT login endpoint
POST	<code>`/api/book`</code>	Guest	<code>`{ roomId, checkIn, checkOut, serviceIds }`</code>	Secures draft reservation & returns reservationId
DELETE	<code>`/api/book`</code>	Guest/Staff	<code>`?id={reservationId}`</code>	Executes tiered refund & updates DB to CANCELED
GET	<code>`/api/checkout/[id]`</code>	Guest	URL param <code>`id`</code>	Returns checkout session summary & total invoice
POST	<code>`/api/checkout/[id]/session`</code>	Guest	<code>`{ guestPhone }`</code>	Creates Dodo checkout & returns hosted payment URL
POST	<code>`/api/webhooks/dodo`</code>	System	Dodo webhook payload	Updates reservation to CONFIRMED & payment to COMPLETED
GET	<code>`/api/dashboard/guest`</code>	Guest	Session JWT	Returns guest bookings, favorites, & profile info
GET	<code>`/api/dashboard/staff`</code>	Staff	Session JWT	Returns housekeeping task list & room statuses
GET	<code>`/api/dashboard/admin`</code>	Admin	Session JWT	Returns system KPIs, audit logs & revenue ledgers
GET	<code>`/api/favorites`</code>	Guest	Session JWT	Returns array of favorite resort IDs
POST	<code>`/api/favorites`</code>	Guest	<code>`{ resortId }`</code>	Toggles bookmark status of resort ID
POST	<code>`/api/admin/roles`</code>	Admin	<code>`{ name, permissions }`</code>	Creates custom operational role entity

POST	`/api/admin/staff`	Admin	`{ email, password, name, roleId, deptId }`	Provisions new staff account & links department
POST	`/api/admin/departments`	Admin	`{ name, manager }`	Creates operational resort department entity
POST	`/api/admin/services`	Admin	`{ name, category, price, resortId }`	Creates add-on service experience entity

10. Environment Variables & Setup Guide

To deploy or run Bookme.com locally, create a .env file in the project root containing:

```
# Database Connection
DATABASE_URL="postgresql://user:password@localhost:5432/resort_db?schema=public"

# NextAuth Authentication Config
NEXTAUTH_SECRET="your-super-secret-jwt-key-change-in-production"
NEXTAUTH_URL="http://localhost:3000"

# Nodemailer SMTP Email Dispatch
SMTP_HOST="smtp.gmail.com"
SMTP_PORT=587
SMTP_USER="your-email@gmail.com"
SMTP_PASS="your-app-specific-password"

# Payment Gateway Keys
DODO_PAYMENTS_API_KEY="test_dodo_api_key_string"
TO_EMAIL="code.faisal.dev@gmail.com"
```

Step-by-Step Command Execution Guide:

1. **Install Dependencies:** npm install
2. **Database Migration Push:** npx prisma db push
3. **Seed Database:** node prisma/seed.js
4. **TypeScript Compiler Audit:** npx tsc --noEmit
5. **Launch Development Server:** npm run dev
6. **Production Build Audit:** npx next build

End of Official Technical System Manual — Bookme.com System Architecture v1.0.0